

Clase 9 – Despliegue Productivo

Oscar Rodríguez

Advanced Time Series Analysis

Noviembre 2025

Mapa de la clase

- 1 Conceptos básicos
- 2 Estructura y configuración
- 3 Implementación del servicio
- 4 Ejecución y pruebas
- 5 Producción y buenas prácticas
- 6 Checklist y cierre

¿Qué es una API?

API (*Application Programming Interface*) es una interfaz para que dos programas se comuniquen de forma estructurada.

- En la web, una **API HTTP** expone *endpoints* que reciben peticiones (GET/POST/...) y devuelven respuestas (JSON).
- Permite separar: *cálculo/modelo* (backend) de *consumo* (front, script, BI).
- En nuestro curso: servimos un **modelo de series de tiempo** (GRU/LSTM) como servicio.

Endpoints y OpenAPI

Endpoint = ruta + método + contrato de datos.

- Ej.: POST /train/ → entrena y guarda gráficos.
- GET /health → diagnóstico rápido.

OpenAPI: especificación estandarizada del contrato.

- FastAPI genera OpenAPI automáticamente.
- **Swagger UI**: /docs. **ReDoc**: /redoc.
- Validación con **Pydantic**: tipos, rangos, valores por defecto.

Mapa de la clase

- 1 Conceptos básicos
- 2 Estructura y configuración**
- 3 Implementación del servicio
- 4 Ejecución y pruebas
- 5 Producción y buenas prácticas
- 6 Checklist y cierre

Estructura mínima del proyecto

- Configurador
- Endpoint
- Data

Sugerencias:

- Mantener `config.json` versionado.
- Separar *carga-preproceso*, *modelo*, *servicio*.
- Usar nombres consistentes de columnas (Fecha, objetivo, features).

config.json (ejemplo)

```
{  
  "path": "/Users/orodriguez/PycharmProjects/TimeSeries/indicadores_causales.csv",  
  "target_column" : "USD_COP",  
  "architecture": [32, 16],  
  "epochs": 300,  
  "days_topredict" : 30  
}
```

Idea: cambiar el comportamiento del servicio sin editar código.

Entorno y dependencias

Conda (recomendado): `conda create -n AdvTimeSeries python=3.12 -y conda activate AdvTimeSeries conda install numpy pandas matplotlib statsmodels -y pip install fastapi uvicorn tensorflow==2.16.2`

Notas prácticas:

- En macOS, para generar PNG en servidor: `matplotlib.use('agg')`.
- Evitar abrir ventanas GUI (`plt.show()`). Usar `plt.savefig(...)`.

Mapa de la clase

- 1 Conceptos básicos
- 2 Estructura y configuración
- 3 Implementación del servicio**
- 4 Ejecución y pruebas
- 5 Producción y buenas prácticas
- 6 Checklist y cierre

Partes de un servicio FastAPI

- 1 **App:** `app = FastAPI(title=..., description=...)`.
- 2 **Modelos Pydantic:** definen el contrato (inputs/outputs).
- 3 **Lógica:** funciones de preproceso, entrenamiento, predicción.
- 4 **Endpoints:** `@app.post("/train/")` que orquestan la lógica.
- 5 **Persistencia:** gráficos/artefactos (grapics/, .h5, .json).

Esqueleto típico del endpoint /train/

```
@app.post("/train/") def *****  
*****  
*****  
return *****
```

Mapa de la clase

- 1 Conceptos básicos
- 2 Estructura y configuración
- 3 Implementación del servicio
- 4 Ejecución y pruebas**
- 5 Producción y buenas prácticas
- 6 Checklist y cierre

Correr en local (desarrollo)

Comando básico: `uvicorn NombredelPy:app --reload`
nombre_archivo.py(sin.py) app = objetoFastAPIdentrodelarchivo

Comprobación:

- Navegar a `http://127.0.0.1:8000/docs` (Swagger UI).
- Pulsar **Try it out** en `POST /train/` y **Execute**.
- Revisar salida JSON y `graphics/*.png`.

Mapa de la clase

- 1 Conceptos básicos
- 2 Estructura y configuración
- 3 Implementación del servicio
- 4 Ejecución y pruebas
- 5 Producción y buenas prácticas**
- 6 Checklist y cierre

De desarrollo a producción

Servidor de aplicación

1 worker por CPU como regla simple: uvicorn

```
api_imeSeries : app --host 0,0,0,0 --port 8000 --workers 4
```

Reverse proxy (Nginx)

- Termina TLS (HTTPS), balancea, comprime, cachea estáticos.
- Redirige / → `http://127.0.0.1:8000`.

Otros: variables de entorno (rutas, claves), logs estructurados (JSON), rotación de logs, límites de tiempo.

Docker (opcional)

Dockerfile mínimo: FROM python:3.12-slim WORKDIR /app COPY requirements.txt ./ RUN pip install -r requirements.txt COPY . . ENV MPLBACKEND=Agg EXPOSE 8000 CMD [üvicorn", ".api_imeSeries : app", " — —host", "0,0,0,0", " — —port", "8000", " — —workers", "2"]
Build & run: docker build -t ts-fastapi . docker run -p 8000:8000 -v PWD/grapics : /app/grapicsts — fastapi

Errores comunes y soluciones

- **Matplotlib/GUI en macOS:** "NSWindow... main thread" $\Rightarrow$ usar backend no interactivo: `import matplotlib; matplotlib.use('.Agg')`
- **Advertencia Keras input_shape:** preferir `tf.keras.Input(shape=...)` como primera capa.
- **FileNotFound:** verificar `config.json` y rutas absolutas al CSV.
- **CORS:** si consumirás desde web, habilitar CORS en FastAPI (orígenes permitidos).

Salidas y artefactos

- **Gráficos:** `grapics/prediccion_{target}.png`,
`grapics/perdida_{target}.png`.
- **Métricas:** MSE de entrenamiento/validación en la respuesta JSON.
- **Persistencia de modelo** (opcional):
`model.save("models/gru_1atest.keras")` luego : `tf.keras.models.load_model(...)`

Seguridad y observabilidad (mínimos)

- **Seguridad:** HTTPS detrás de Nginx, uvicorn `-proxy-headers`, limitar `/docs` en producción si es necesario.
- **Rate limiting:** a nivel proxy (Nginx) o middleware.
- **Logs:** estructurados; registrar input clave y tiempos de entrenamiento.
- **Salud:** endpoint `/health` que devuelva `{"status": ".ok"}`.
- **Trazas/Métricas:** Prometheus/Grafana (opcional) para tiempos y errores.

Mapa de la clase

- 1 Conceptos básicos
- 2 Estructura y configuración
- 3 Implementación del servicio
- 4 Ejecución y pruebas
- 5 Producción y buenas prácticas
- 6 Checklist y cierre

Checklist de despliegue

- 1 Crear entorno e instalar dependencias.
- 2 Preparar config.json con ruta, objetivo, arquitectura, epochs, horizonte.
- 3 Verificar api_TimeSeries.py: `matplotlib.use('.agg')`, `Input(shape=...)`.
- 4 Lanzar en local: `uvicorn ... -reload`.
- 5 Probar en /docs; revisar graphics/.
- 6 Opcional: empaquetar con Docker y servir detrás de Nginx.
- 7 Monitorizar logs y espacio en disco (gráficos/modelos).

Comandos clave

Local (dev) `uvicorn apiTimeSeries : app --reload`

Producción simple (múltiples workers) `uvicorn`

`apiTimeSeries : app --host 0,0,0,0 --port 8000 --workers 4`

`curl` (usar config.json por defecto) `curl -X POST http://127.0.0.1:8000/train/`

`curl` (sobrescribir parámetros) `curl -X POST http://127.0.0.1:8000/train/ -H "Content-Type: application/json" -d`

`'{ "architecture": [64,32], "epochs": 150, "daystopredict"` : 15'

Conclusión

FastAPI permite exponer **modelos de series de tiempo** como servicios web fiables:

- Definir claramente **endpoints** y contratos (Pydantic/OpenAPI).
- Separar **configuración** en `config.json`.
- Ejecutar con Uvicorn; endurecer con Nginx/Docker para producción.
- Registrar **métricas**, **artefactos** y errores de forma trazable.

Con esto, un pipeline de GRU/LSTM pasa de notebook a **servicio reproducible en producción**.